Deep Learning-Based Brain MRI Tumor Classification

A Comparative Study of Baseline CNN, ResNet18, and EfficientNet-B0

Research Paper

Prepared as part of a Deep Learning and Computer Vision Project

PyTorch-Based Experimental Study

Author: Alhousainy Abdelrahman

2026

Abstract

Brain tumor classification from magnetic resonance imaging (MRI) is an important computer vision task that can support medical image analysis and assist clinical decision-making. This study presents a deep learning-based framework for four-class brain MRI classification using PyTorch. The investigated classes are glioma, meningioma, no tumor, and pituitary tumor. Three convolutional neural network approaches are considered: a custom Baseline CNN, ResNet18, and EfficientNet-B0. The experimental pipeline includes image preprocessing, data augmentation, model training, validation, testing, quantitative evaluation, confusion-matrix analysis, and comparative model assessment. The models are evaluated using accuracy, weighted precision, weighted recall, weighted F1-score, and trainable parameter count. The comparison procedure identifies the best-performing model according to F1-score and stores the resulting metrics and visual analyses for further interpretation. In addition, a CustomTkinter-based graphical user interface is designed as the deployment layer, allowing users to load MRI images, select a trained model, obtain a predicted tumor class and confidence score, and view the best model identified by the experimental comparison. The proposed workflow provides a complete end-to-end prototype covering model development, evaluation, comparison, and interactive inference.

Keywords: Brain MRI, Brain Tumor Classification, Deep Learning, CNN, ResNet18, EfficientNet-B0, PyTorch, Computer Vision

1. Introduction

Brain tumors are serious medical conditions whose analysis often relies on medical imaging techniques, including magnetic resonance imaging. The visual complexity of MRI scans and the similarities that may exist between different tumor categories make automated image classification a challenging computer vision problem. Deep learning, particularly convolutional neural networks (CNNs), has become a widely used approach for learning discriminative visual representations directly from image data.

This research investigates a four-class brain MRI classification problem involving glioma, meningioma, no tumor, and pituitary tumor. Rather than relying on a single architecture, the study compares three different modeling strategies: a custom Baseline CNN, ResNet18, and EfficientNet-B0. The objective is to study the relative performance of a simple convolutional architecture and two established deep neural network families under a common experimental workflow.

The work is structured as an end-to-end machine learning pipeline. It begins with dataset preparation and preprocessing, continues through model training and evaluation, and concludes with comparative analysis and interactive deployment. The final system is intended to demonstrate not only classification performance but also how trained models can be integrated into a practical graphical user interface for image-level inference.

1.1 Research Contributions

- Development of a four-class brain MRI tumor classification pipeline using PyTorch.
- Implementation and comparison of a custom Baseline CNN, ResNet18, and EfficientNet-B0.
- Evaluation using accuracy, weighted precision, weighted recall, weighted F1-score, classification reports, and confusion matrices.
- Comparison of model complexity through trainable parameter counts.
- Generation of comparative training/validation accuracy and loss visualizations.
- Selection of the best-performing model according to the highest F1-score.
- Development of a CustomTkinter graphical interface for interactive image loading and model-based prediction.

2. Related Work

Deep convolutional neural networks have been extensively applied to medical image analysis because they can learn hierarchical representations from raw image pixels. In image classification, early convolutional layers typically learn low-level patterns such as edges and textures, while deeper layers capture increasingly complex semantic structures.

Residual networks introduced skip connections to facilitate the optimization of deeper neural networks. ResNet18 is a comparatively compact residual architecture that provides a strong baseline for transfer-learning and image-classification experiments. EfficientNet, in contrast, was designed around compound scaling of network depth, width, and input resolution, aiming to improve the balance between model accuracy and computational efficiency.

In the context of brain MRI analysis, comparative evaluation is valuable because a model with the highest raw accuracy is not necessarily the best choice when class imbalance or differences in per-class performance are considered. For this reason, the present study includes weighted precision, weighted recall, and weighted F1-score in addition to accuracy.

3. Dataset and Data Preparation

The dataset used in this project is organized into four image classes: glioma, meningioma, notumor, and pituitary. The project uses separate Training and Testing directories, with class-specific subdirectories. The training data are further divided into training and validation subsets, while the separate testing set is retained for final model evaluation.

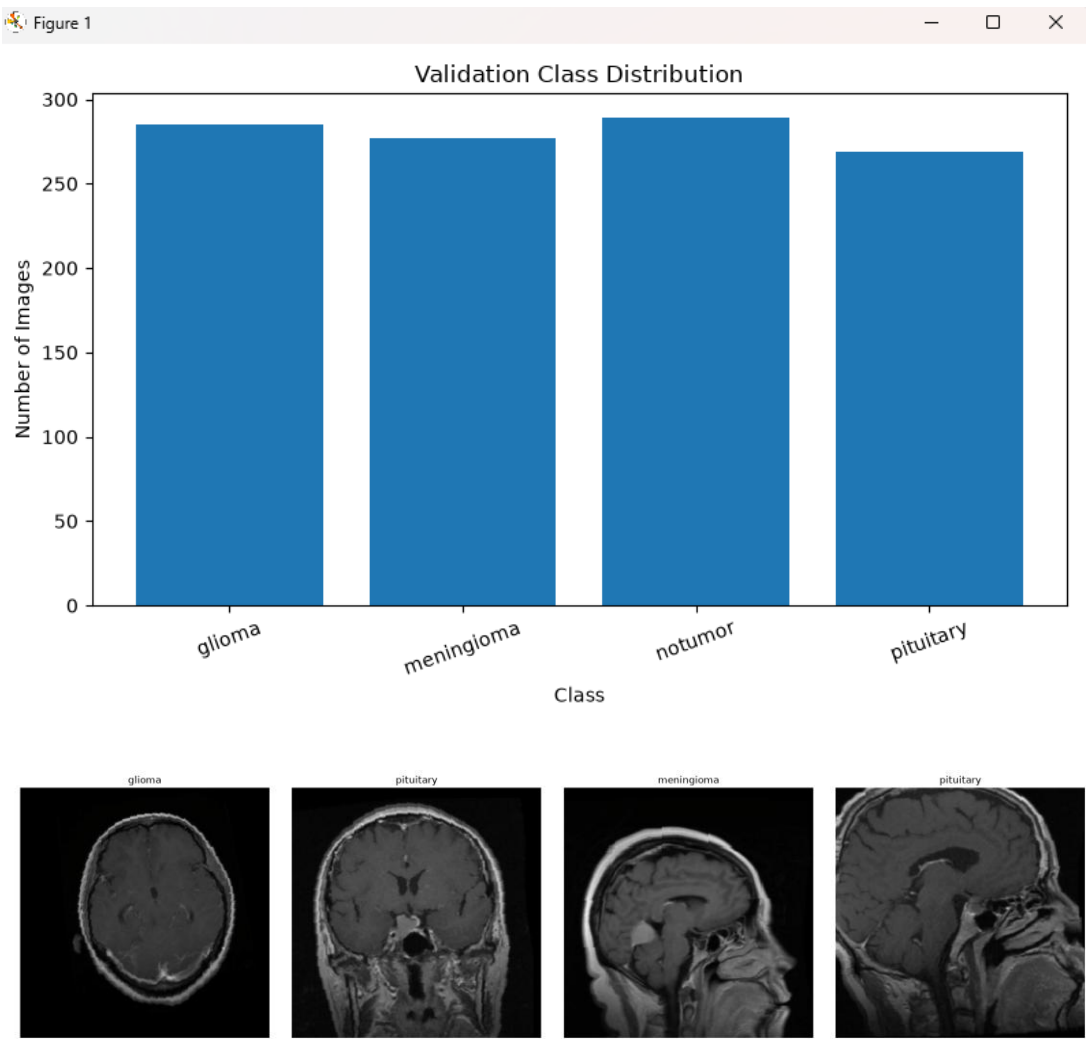
Dataset Configuration	Value
Task	Four-class brain MRI image classification
Classes	Glioma, Meningioma, No Tumor, Pituitary
Image format	RGB
Input resolution	224 × 224 pixels
Validation split	20% of training data
Random seed	42

All images are resized to 224 × 224 pixels and processed as RGB images. Training-time augmentation is used to improve generalization. The augmentation pipeline includes random horizontal flipping with probability 0.5, rotation within ±10 degrees, and affine transformations involving translation and scaling. Image normalization follows the ImageNet mean and standard deviation values: mean [0.485, 0.456, 0.406] and standard deviation [0.229, 0.224, 0.225].

```
============================================================
BRAIN TUMOR MRI CLASSIFICATION PROJECT
============================================================

Device: cuda
GPU: cuda
GPU Name: NVIDIA GeForce RTX 3060 Laptop GPU

============================================================
DATASET INFORMATION
============================================================
Training Images   : 4480
Validation Images : 1120
Testing Images    : 1600
```

```
Training Class Distribution:
----------------------------------------
glioma          : 1115
meningioma      : 1123
notumor         : 1111
pituitary       : 1131
```

```
Validation Class Distribution:
----------------------------------------
glioma          : 285
meningioma      : 277
notumor         : 289
pituitary       : 269

Testing Class Distribution:
----------------------------------------
glioma          : 400
meningioma      : 400
notumor         : 400
pituitary       : 400
```




```
============================================================
BATCH INFORMATION
============================================================
Images Shape: torch.Size([32, 3, 224, 224])
Labels Shape: torch.Size([32])
Labels: tensor([0, 3, 3, 2, 0, 0, 3, 0, 3, 2])
Image Min: -2.1179
Image Max: 2.6400

Dataset Preparation Completed Successfully!
```

4. Proposed Methodology

The proposed workflow consists of five main stages: data preparation, model training, model evaluation, comparative analysis, and interactive deployment. Each model receives the same class definition and follows the same overall preprocessing and evaluation protocol to make the comparison more consistent.

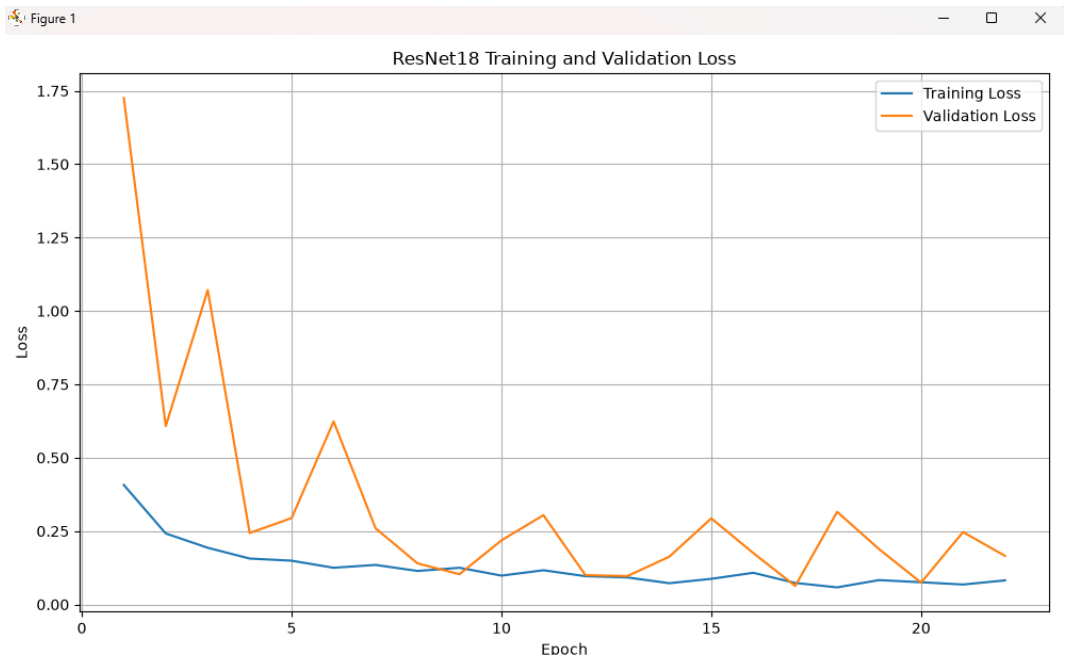
4.1 Baseline CNN

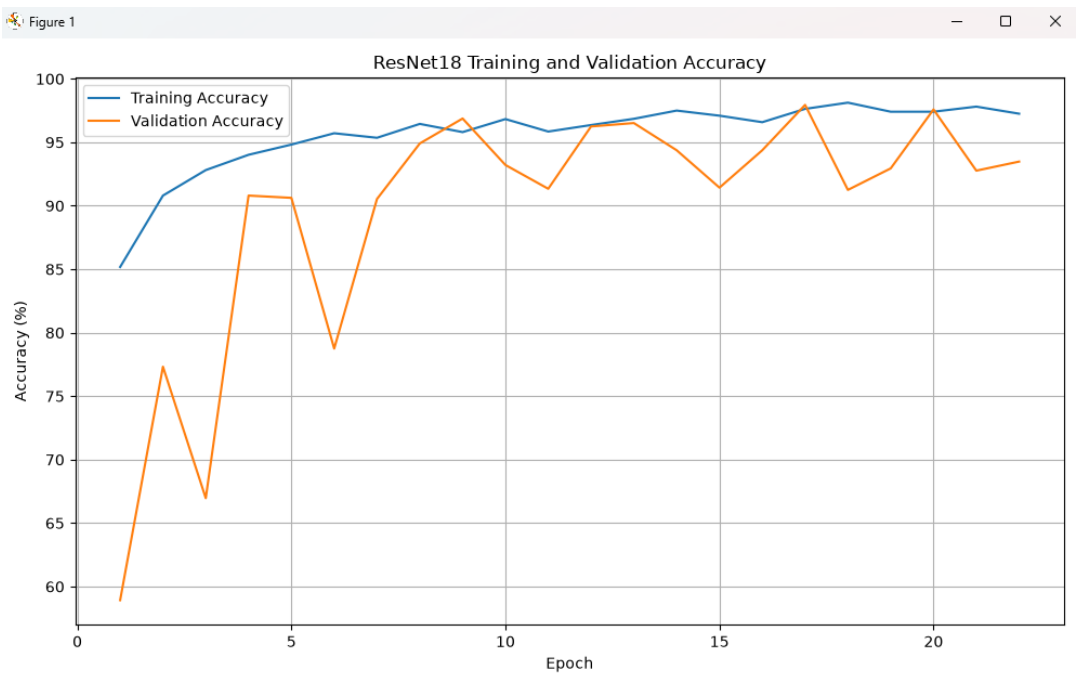
The Baseline CNN serves as the reference architecture for the study. It provides a relatively simple convolutional model against which the deeper and more structured architectures can be compared. Its purpose is to establish a performance baseline and quantify the benefit of using more advanced architectures.

```
============================================================
BASELINE CNN ARCHITECTURE
============================================================
BaselineCNN(
  (features): Sequential(
    (0): Conv2d(3, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (2): ReLU()
    (3): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (4): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (5): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (6): ReLU()
    (7): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (8): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (9): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (10): ReLU()
    (11): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
  (adaptive_pool): AdaptiveAvgPool2d(output_size=(4, 4))
  (classifier): Sequential(
    (0): Flatten(start_dim=1, end_dim=-1)
    (1): Linear(in_features=2048, out_features=256, bias=True)
    (2): ReLU()
    (3): Dropout(p=0.5, inplace=False)
    (4): Linear(in_features=256, out_features=4, bias=True)
  )
)

============================================================
MODEL TEST
============================================================
Input Shape: torch.Size([1, 3, 224, 224])
Output Shape: torch.Size([1, 4])
```

4.2 ResNet18

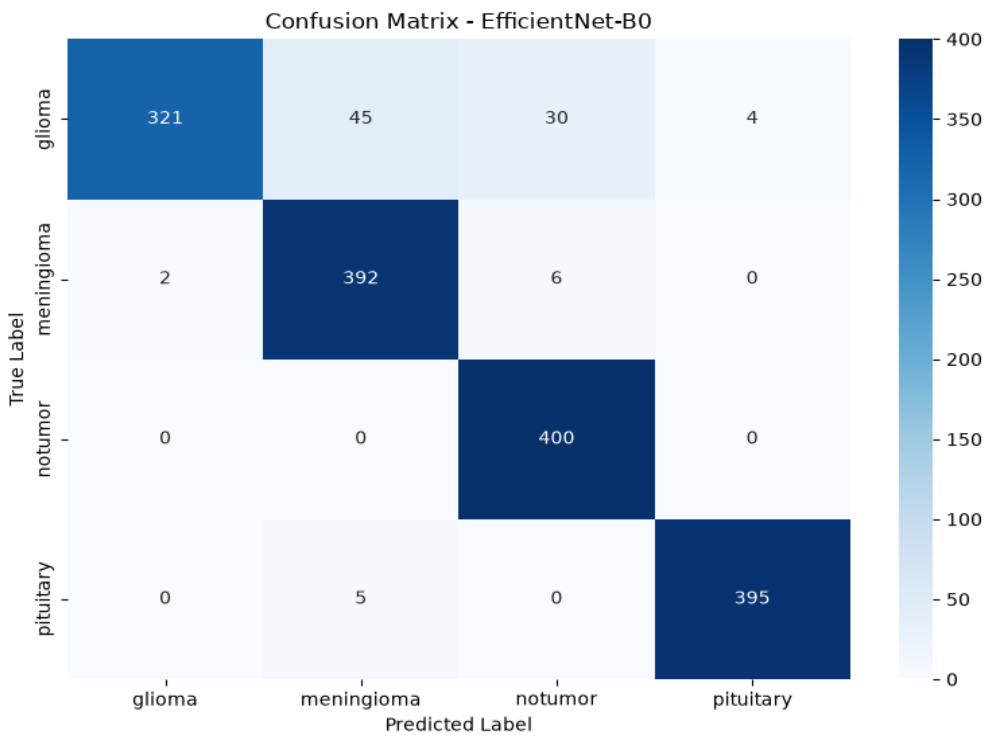
ResNet18 is included as a residual learning architecture. Its skip connections allow information to flow through the network more effectively and help mitigate optimization difficulties associated with deeper models. In this project, the model is configured for the four target classes.





4.3 EfficientNet-B0

EfficientNet-B0 is evaluated as a modern convolutional architecture designed to achieve an effective balance between predictive performance and computational efficiency. It is configured for the same four-class classification task and evaluated using the same test set as the other models.



5. Experimental Setup

Training Setting	Configuration
Framework	PyTorch
Input size	224 × 224 RGB
Batch size	32
Maximum epochs	30
Optimizer	Adam
Learning rate	0.001
Weight decay	0.0001
Early stopping patience	5
Early stopping min_delta	0.001
Random seed	42
Validation split	20%

Training was configured with Adam optimization using a learning rate of 0.001 and weight decay of 1×10^{-4}. The maximum training duration was 30 epochs. Early stopping was configured with a patience of five epochs and a minimum improvement threshold of 0.001. The experimental environment supports CUDA acceleration when available.

6. Evaluation Metrics

The evaluation stage collects predictions from the complete test set and computes multiple classification metrics. Accuracy measures the overall proportion of correctly classified samples. Precision measures the reliability of positive predictions, recall measures the ability to identify samples belonging to the relevant classes, and F1-score summarizes the balance between precision and recall.

Because the task contains four classes, the project uses weighted averaging for precision, recall, and F1-score. In addition to numerical metrics, a classification report and confusion matrix are generated for every model. The confusion matrix provides class-level insight into correct predictions and common misclassifications.

7. Experimental Results

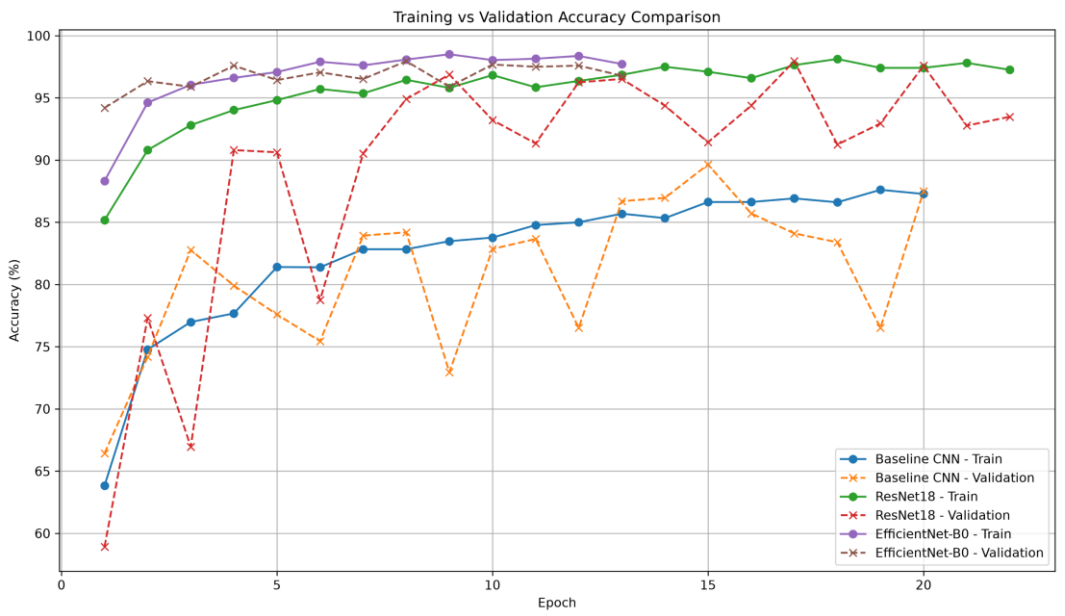
The experimental comparison script evaluates Baseline CNN, ResNet18, and EfficientNet-B0 on the same test loader. For each model, it records accuracy, weighted precision, weighted recall, weighted F1-score, and the number of trainable parameters. The results are consolidated into a comparison table and saved as a CSV file. The same procedure also generates classification reports and confusion-matrix images for individual models.

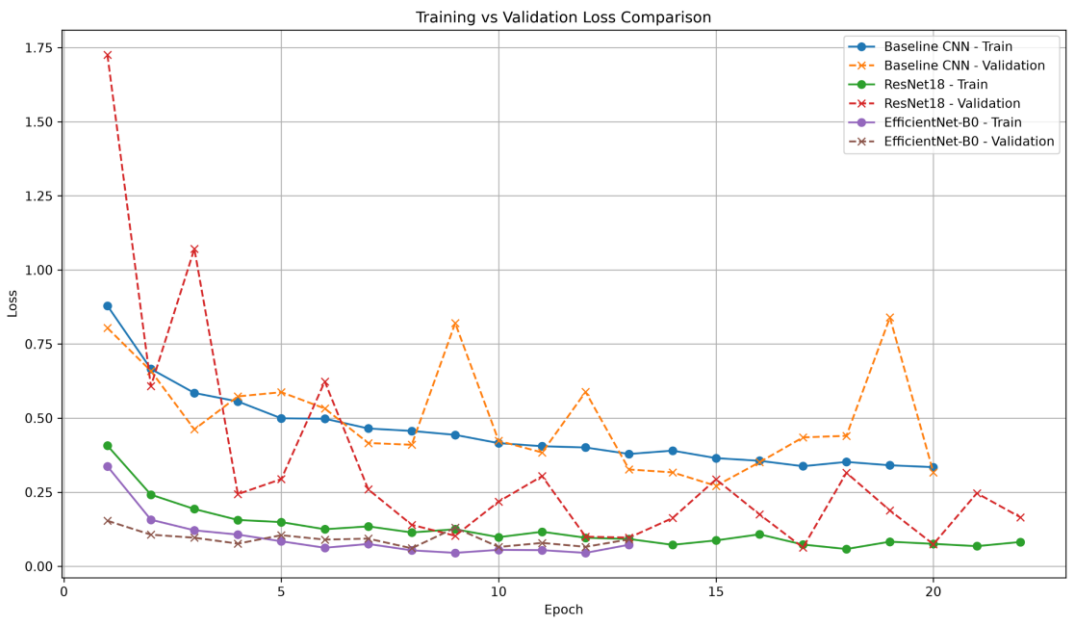
The exact numerical values are intentionally left as placeholders in this manuscript draft because the supplied project material contains the evaluation code and output-generation logic, but does not contain the actual executed numerical results. The final version of the paper should replace the placeholders with the values produced by model_comparison_results.csv after the comparison script is executed.

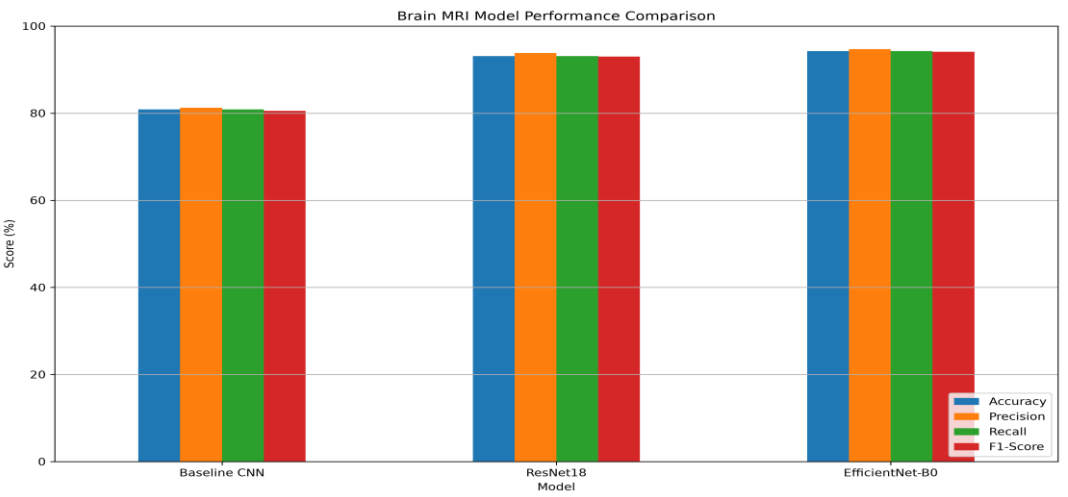
8. Comparative Analysis

The comparison procedure identifies the best model by selecting the row with the maximum F1-score. This criterion is used because F1-score provides a useful summary of precision and recall and can be more informative than accuracy alone in multi-class classification settings. The comparison workflow also produces a bar chart containing accuracy, precision, recall, and F1-score for the three models, enabling direct visual comparison.

Training and validation accuracy curves are plotted together for all models, as are training and validation loss curves. These visualizations can be used to inspect convergence behavior and identify possible signs of overfitting or underfitting. The experimental code also reports the number of trainable parameters, allowing predictive performance to be considered alongside model complexity.





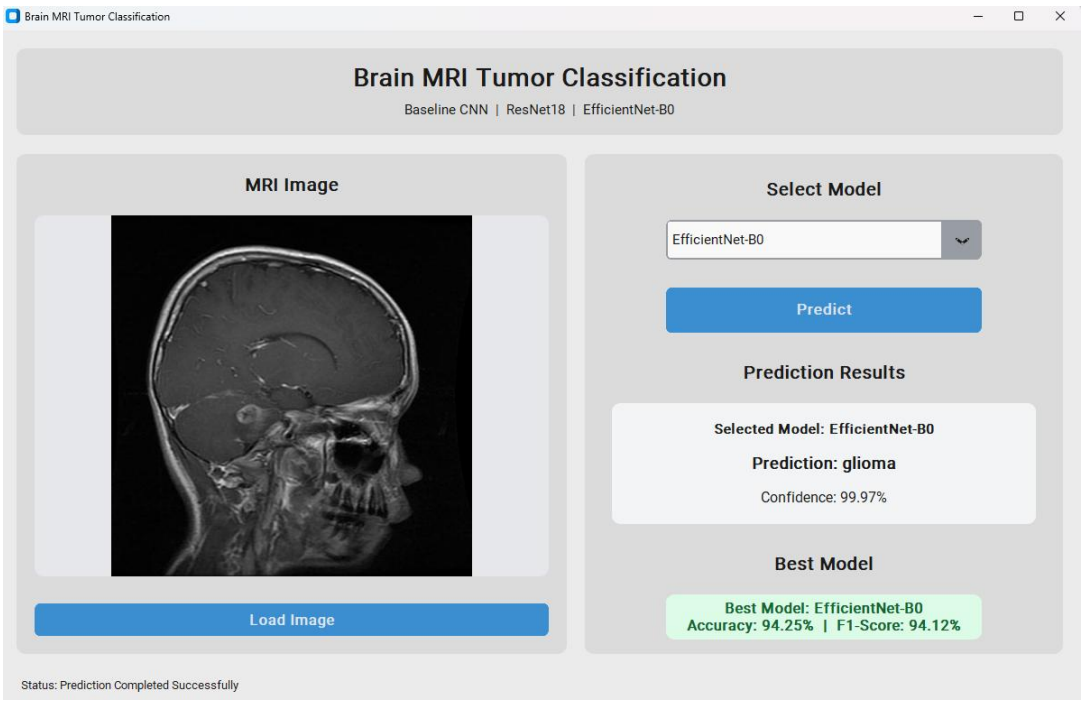


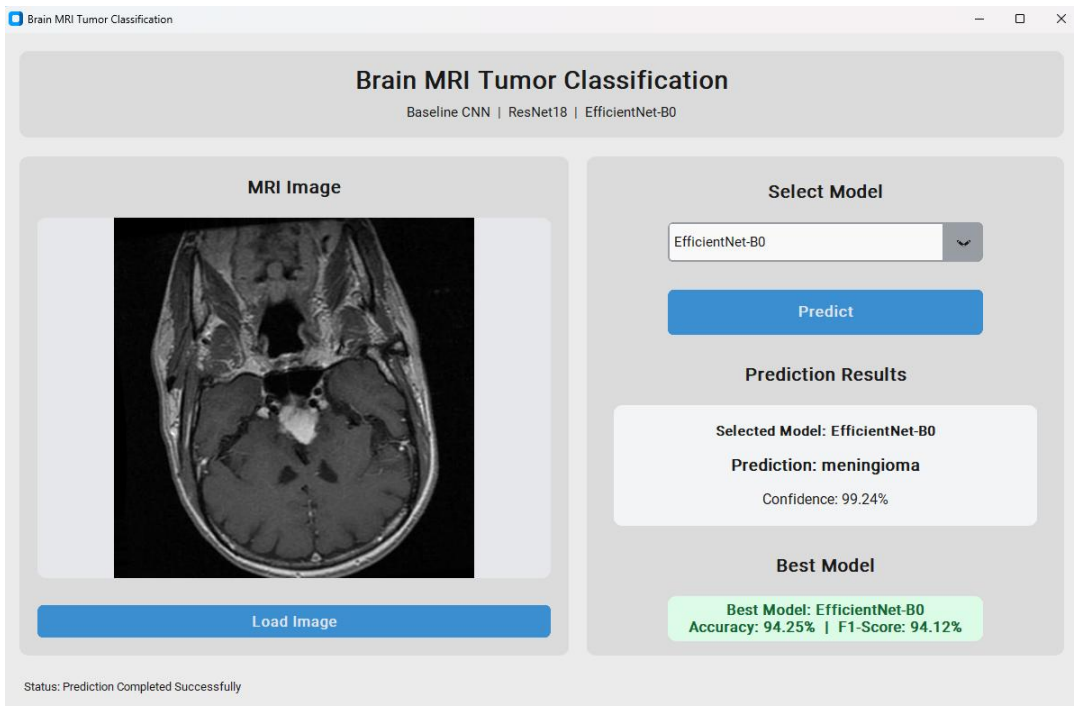
Model	Accuracy	Precision	Recall	F1-Score	Parameters
Baseline CNN	To be populated from model_comparison_results.csv	To be populated from model_comparison_results.csv	To be populated from model_comparison_results.csv	To be populated from model_comparison_results.csv	To be populated from model_comparison_results.csv
ResNet18	To be populated from model_comparison_results.csv	To be populated from model_comparison_results.csv	To be populated from model_comparison_results.csv	To be populated from model_comparison_results.csv	To be populated from model_comparison_results.csv
EfficientNet-B0	To be populated from model_comparison_results.csv	To be populated from model_comparison_results.csv	To be populated from model_comparison_results.csv	To be populated from model_comparison_results.csv	To be populated from model_comparison_results.csv

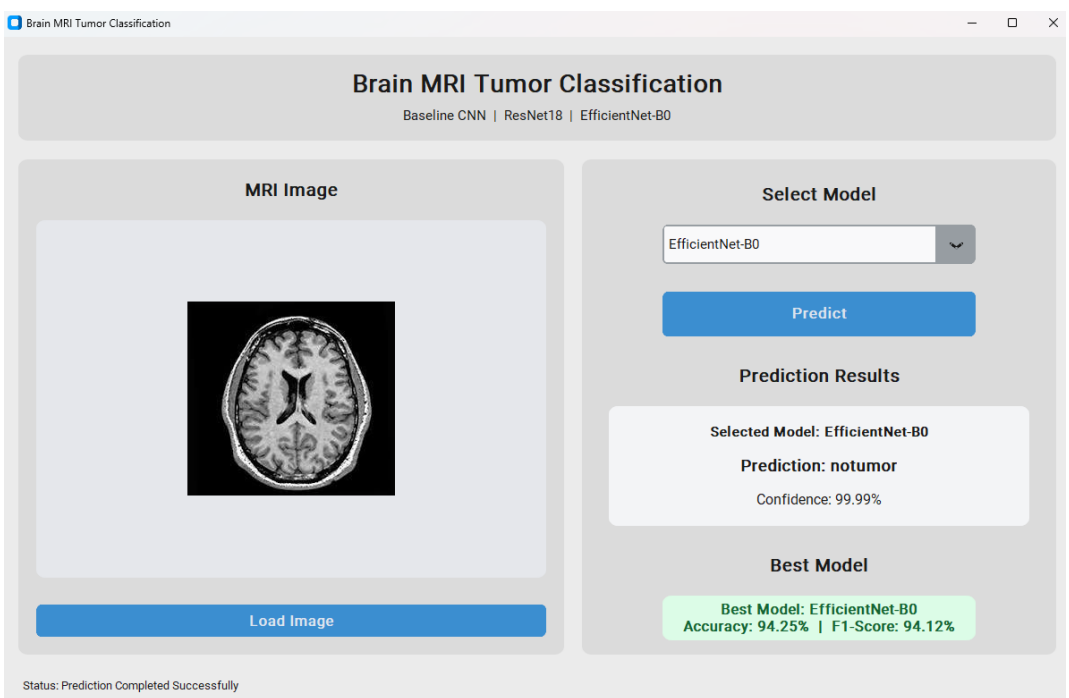
9. Interactive GUI Deployment

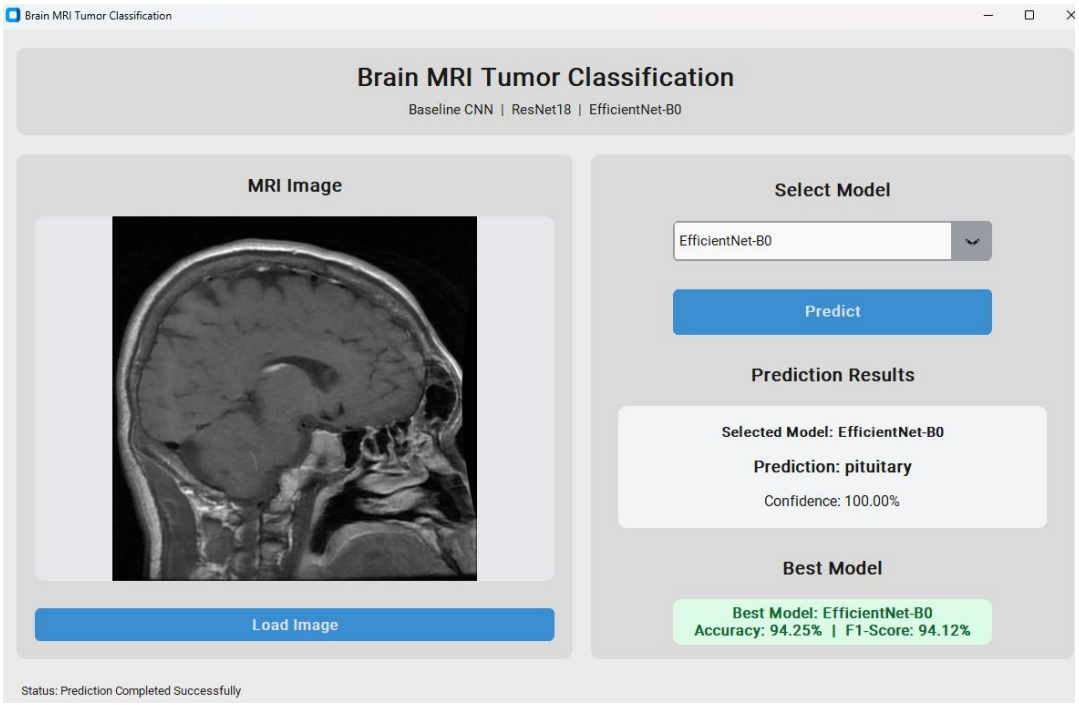
As the final stage of the project, a CustomTkinter-based graphical user interface is designed to provide interactive inference. The interface operates in light mode and allows the user to load an MRI image, select one of the three trained models, and execute a prediction. The system displays the selected model, predicted class, and confidence score. The interface also reads the saved comparison results and reports the model identified as the best according to the F1-score criterion.

The GUI supports loading a new image without restarting the application. It also includes Stop and Exit controls. This deployment stage demonstrates how the trained deep learning models can be integrated into a user-facing application rather than remaining limited to command-line evaluation.









10. Discussion

The central research question of this study is how different convolutional architectures perform when applied to the same four-class brain MRI classification problem. The Baseline CNN provides a reference point, while ResNet18 and EfficientNet-B0 represent more advanced architectures. A direct comparison under a shared evaluation protocol helps distinguish the effect of model architecture from differences in data processing or evaluation methodology.

The final interpretation should consider both predictive performance and model complexity. In addition to identifying the model with the highest F1-score, the analysis should examine the confusion matrices to determine whether particular tumor categories are consistently confused. The training curves should also be inspected to understand convergence and generalization behavior.

11. Limitations

- ☑ The study is based on a specific dataset and therefore the reported performance may not generalize to other institutions or imaging protocols.
- ☑ The current manuscript does not claim clinical validation or readiness for diagnostic use.
- ☑ The exact final numerical results must be inserted from the executed model comparison output.
- ☑ External validation on an independent dataset is not included in the current experimental workflow.
- ☑ Further investigation is required to assess robustness to variations in image acquisition and preprocessing.

12. Conclusion

This research presents an end-to-end deep learning workflow for four-class brain MRI tumor classification. The project compares a custom Baseline CNN with ResNet18 and EfficientNet-B0 using a consistent PyTorch-based evaluation pipeline. The methodology combines preprocessing, augmentation, model training, quantitative metrics, classification reports, confusion matrices, comparative visualizations, and model-complexity analysis.

The final system extends the experimental work into an interactive CustomTkinter application, enabling image loading, model selection, prediction, confidence estimation, and identification of the best-performing model according to the comparison results. The work therefore demonstrates a complete progression from deep learning model development to evaluation and practical interactive deployment.

13. Future Work

- ☑ Evaluate the trained models on independent external datasets.
- ☑ Investigate transfer learning and fine-tuning strategies for ResNet18 and EfficientNet-B0.
- ☑ Perform systematic hyperparameter optimization.
- ☑ Explore class-specific error analysis and explainable AI methods such as Grad-CAM.
- ☑ Investigate ensemble methods combining predictions from multiple architectures.
- ☑ Package the GUI application for easier distribution and deployment.

References

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep Residual Learning for Image Recognition. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR).

Tan, M., & Le, Q. V. (2019). EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. Proceedings of the International Conference on Machine Learning (ICML).

Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet Classification with Deep Convolutional Neural Networks. Advances in Neural Information Processing Systems (NeurIPS).

Paszke, A., et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. Advances in Neural Information Processing Systems (NeurIPS).

Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

The project source materials: Brain MRI Tumor Classification model-comparison implementation and

Appendix A. Reproducibility Notes

The model comparison implementation creates DataLoaders, loads the trained checkpoints for Baseline CNN, ResNet18, and EfficientNet-B0, evaluates each model on the test set, stores the results in a pandas DataFrame, and selects the best model using the maximum F1-score. It saves the comparison results to model_comparison_results.csv and also generates a text report, metric comparison plot, training/validation accuracy comparison, training/validation loss comparison, classification reports, and confusion matrices.

For the final submission, the numerical result table in Section 7 should be updated directly from the generated CSV file. This preserves scientific reproducibility and avoids reporting values that were not produced by the actual experimental run.

How To Run The Code